

一、摘要

本專題設計並實作一套「智慧行程排程代理系統」，結合大型語言模型 (LLM) 與檢索增強生成 (RAG) 技術。系統前端採用 HTML / CSS / JavaScript 與 Socket 構建即時聊天室與行事曆介面；後端則以 Node.js、MongoDB 及 Python 串接 GPT API (gpt-4o-mini)，負責資料儲存、格式轉換、排程語句判斷、核心排程運算。

本系統具備的主要特色：

1. **意圖分析與分類**：透過兩階段 AI 模型，精準辨識聊天室對話中的排程需求與使用者意圖。
2. **個人化排程推薦**：分析使用者的歷史紀錄，利用統計演算法與多項權重參數，推薦最符合個人習慣的行程時段。
3. **RAG 記憶增強機制**：在進行上述意圖判斷與排程推薦時，系統會檢索歷史相似情境 (向量資料) 作為參考。這不僅能提高 LLM 判斷的準確度，更能動態微調演算法權重，使推薦結果隨時間越來越貼近使用者喜好。

關鍵詞：大型語言模型(LLM)、檢索增強生成(RAG)、智慧排程、前端 (HTML / CSS / JavaScript)、後端(Node.js / Express)、資料庫(Mongodb)

二、專題研究動機與目的

在快節奏的日常生活中，良好的行程安排與時間管理是可以有效的提升人們的做事效率。然而，傳統的行事曆與任務管理工具往往需要使用者自己去安排時間，因此如果使用者不太了解自己的做事習慣，而導致時間管理只是做白工沒有效率，進而造成行程完成度不高與做事仍然效率低下的問題，也可能導致往後行程的延後。

基於上述問題，本專題旨在開發一套的智慧排程系統。我們的目的是讓使用者安排的排程完成度能提高，以此提高做事效率。讓使用者只需像日常聊天般輸入「想做的事情」，系統便能幫助其解析需求，並根據歷史數據推薦較適合的時段，經由此套系統，提高使用者自己的時間管理效率。

三、團隊合作方式

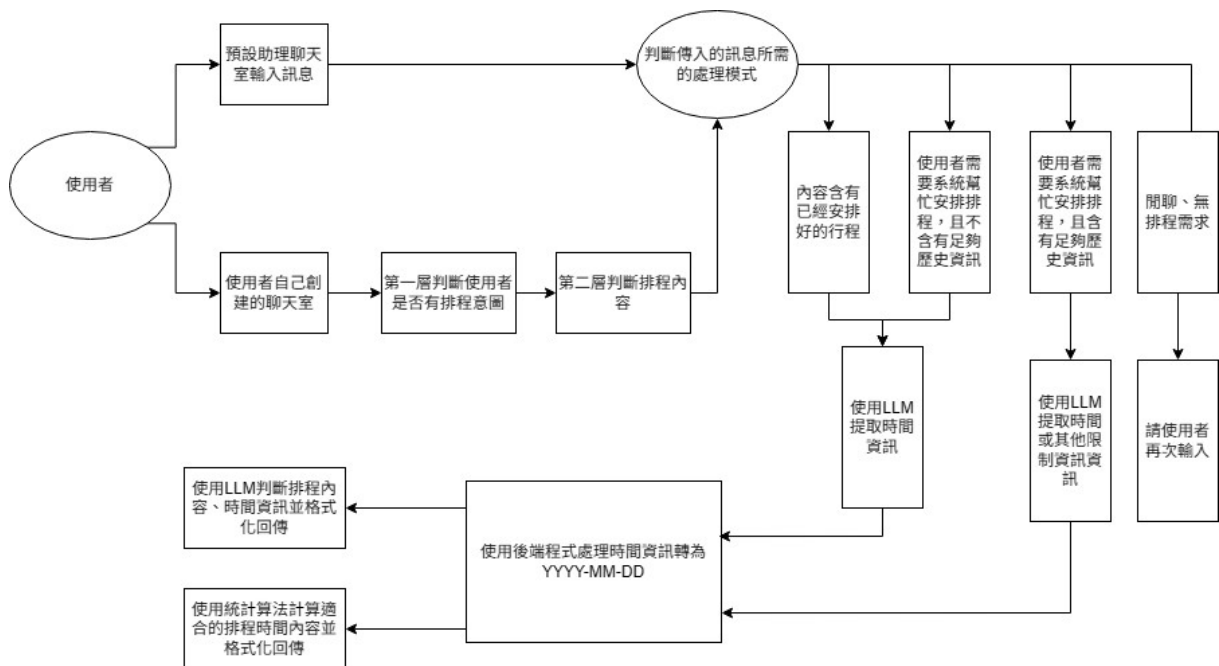
1. **初期**：架構設計與基礎建設 在共同確立系統雛形與核心功能後，即進行分工拆分為：
 - a. 負責建置前端的初始架構與使用者介面
 - b. 負責處理後端伺服器的基礎建設與 LLM (大型語言模型) 的初步串接。
 - c. 雙方根據各自的需求串接或提供 API，建立起系統的前後端基礎功能測試。
2. **中期**：核心演算法與系統架構、資料整合，針對不同情境的排程困難點進行分別處理：
 - a. 負責整體系統架構初始建構，並負責資料前處理，確保前後端的資料流動與演算法能對接。
 - b. 負責開發個人化排程推薦的統計，演算法所需的資料庫串接。

3. **後期**：監聽、分類系統測試與排程演算法測試、AI 優化以及最終系統架構共同實踐，測試各種實際的排程情境來檢驗系統效果。根據演算法結果，微調其中的各項參數權重，根據監聽與分類結果優化傳遞給 AI 的提示詞或 RAG 情境，進而提升系統意圖分析與時間推薦效果。

四、設計原理、研究方法與步驟

本專題在系統設計上，針對「個人助理」與「多人群組」兩種情境進行處理，並採用多個 AI 模組確保意圖識別的準確性，為應對不同情境的排程需求，系統的資料流劃分為兩大輸入管道。

1. 設計邏輯、流程



(圖一)、內部流程圖

(一) AI 助理聊天室 (私人對話情境)

在此情境中，使用者直接與 AI 助理進行一對一互動。系統會將使用者輸入的自然語言交由 AI 決策層進行意圖判定，並依據輸入內容的完整程度與使用者歷史資料狀態，將請求歸類為以下四種處理模式 (Mode)：

Mode 1：明確排程需求

使用者提供清楚且具體的時間與任務目的 (例如：明天下午一點要開會、下禮拜一晚上七點要健身)，可以透過 LLM 加上程式碼上邏輯時間的轉換，直接提取出時間與內容。

Mode 2：模糊排程需求 (歷史資料充足)

使用者提出排程需求，但未指定明確時間 (例如：明天下午要開會、下禮拜一要健身)。且使用者歷史紀錄已經超過 10 筆，則可以開始使用的統計算法。避免在資料過少時開始使用統計的方式，資料量真的過於太少排程重複無變化，進而連帶影響後面排程資料量繼續偏頗。

Mode 3：模糊排程需求 (冷啟動情境)

與 Mode 2 邏輯相似，但當使用者歷史資料不足時，系統無法單純依賴統計推論。此時會結合使用者事先填寫的偏好表單，以及當前對話內容，交由 LLM 進行綜合推論，以避免我們透過算法進行過度推論。

Mode 4：非排程相關對話

若系統判定使用者輸入內容與排程無直接關聯，則不進行任務建立，並引導使用者重新輸入具體需求，以避免系統行程資料遭到無意義資訊干擾。

(二) 普通聊天室 (多人群組情境)

相較於私人對話，群組聊天中同時存在大量閒聊與非結構化資訊，若對每一則訊息皆進行完整排程解析，不僅會造成計算資源浪費，也容易產生誤判。因此，本系統針對群組情境處理方式為兩階段 AI 判斷機制：

第一階段：排程意圖偵測

系統提供少量三句的聊天室歷史紀錄加上使用者發送的訊息，以及 RAG 的語句判斷情境，來判定該語句是否具有潛在排程意圖，並僅回傳 YES 或 NO 作為篩選結果。

第二階段：排程內容萃取

當第一階段判定結果為 YES 時，系統才會啟動第二階段，結合聊天室十句歷史對話，提取實際被討論或決議的任務內容與時間資訊。

此設計可有效提高判斷成功率，並避免當排程內容過於密集時，反覆產生同一結果，或者提取效果不佳。

2. AI 模組之設計原理與方法

為了實現上述多情境、多模式的排程判斷流程，本系統於 AI 決策層中設計多個具備明確職責的處理模組，並廣泛運用 RAG(Retrieval-Augmented Generation) 技術來輔助語意理解：

(一) 私人對話模式分類機制

系統首先需判斷使用者輸入屬於何種排程模式。為降低 AI 判斷的模糊性，本系統建立約 300 筆具標註結果的語句樣本，並將其轉換為語意向量儲存。當使用者輸入訊息時，系統會先檢索語意相近的範例，作為上下文提示提供給語言模型，藉此提升模式分類的穩定度與一致性。

(二) 任務資訊結構化解析方法

當使用者被判定為明確排程需求時，系統會進一步對語句進行實體抽取，將自然語言中的「任務名稱」、「日期」、「時間」與「任務持續時間」等資訊轉換為後端可直接處理的結構化資料格式，確保解析結果能符合系統資料庫格式需求。

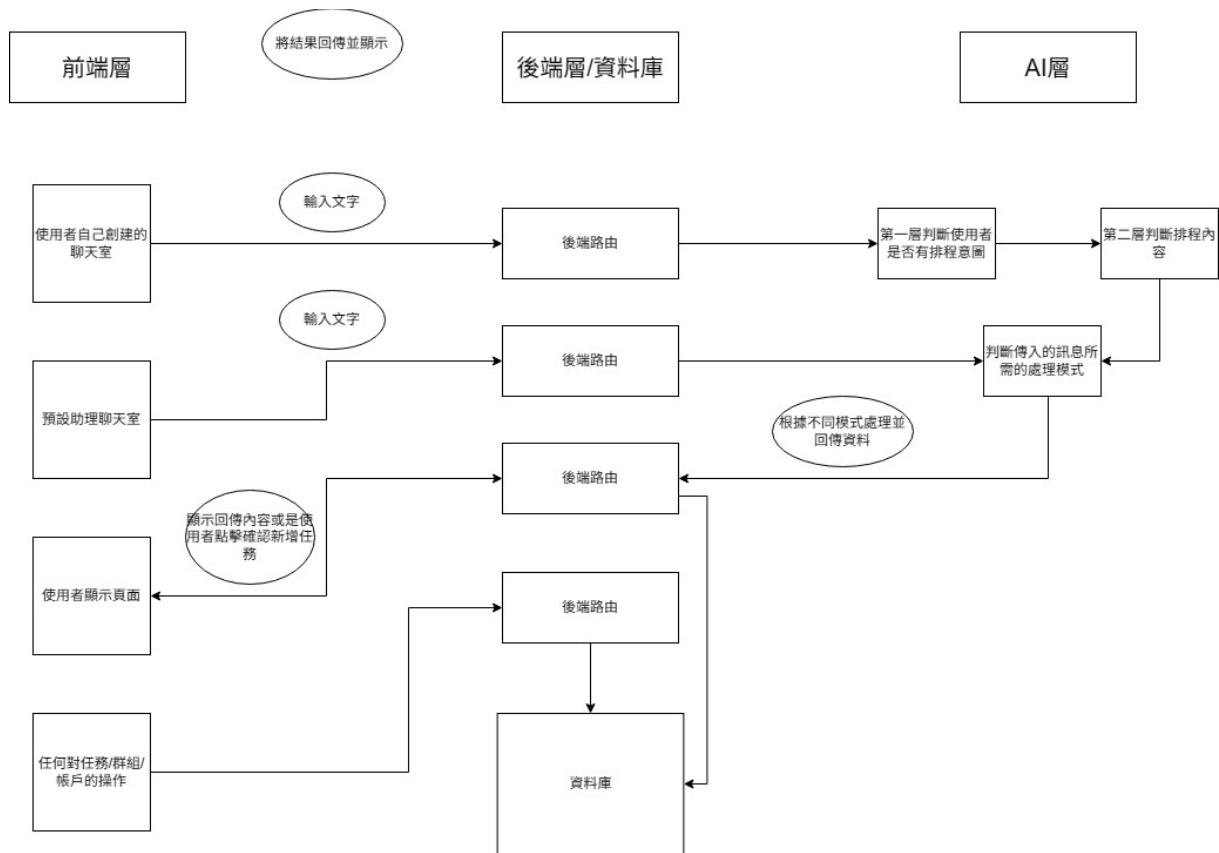
(三) 群組對話兩階段推論策略

在群組聊天中，系統於第一階段僅進行是否具排程意圖的快速判斷，並特別標示目標語句，同時附加近期對話作為語境輔助，以及與(私人對話模式)一樣使用 RAG 提升判斷準確率。僅當確認具有排程意圖時，才進入第二階段，將較長的對話脈絡一次性提供給 AI，協助其從多人對話中整合出最終排程結論，此分層設計可有效降低誤判率，避免當討論行程、任務時，只使用一層來判斷排程訊息太過頻繁費時，並提升系統在實際多人使用情境下的可行性。

(四) 時間資訊提取

在本系統中，最常見的問題是，AI 對於時間的幻覺、過度推論，因此我們採取的做法為要求 AI 提取出使用者輸入的時間字串像是今天、明天、這禮拜 X、下禮拜 X、下個月...、這周末等等語句，並利用 python 程式碼直接處理邏輯回傳準確的日期範圍(YYYY-MM-DD,YYYY-MM-DD)，避免 AI 在時間上做過度的推論、分析。

五、系統架構與實現



(圖二)、整體架構圖

A. 前端層 (Frontend Layer)：負責構建直覺的使用者介面與互動。主要包含兩種聊天室模式，將使用者的輸入路由至不同的處理流程，並顯示最終結果：

- 1. AI 助理聊天室 (私人對話)：**使用者輸入訊息後，系統會透過 API 傳遞至 AI 層進行意圖辨識。AI 生成排程與時間推薦後，成果會直接顯示於此畫面，供使用者一鍵將其新增至個人行事曆。
- 2. 群組聊天室 (多人協作)：**系統會在背景監聽使用者發送的訊息，這些訊息會經過 AI 篩選與判斷。若偵測到與排程相關的內容，系統不會直接在群組干擾對話，而是將排程建議私下推播至該使用者的「AI 助理聊天室」中作後續確認。

B. 後端層 (Backend Layer)：負責前後端資料跨接與邏輯處理。當接收到前端請求後，後端會判斷指令性質，進行 MongoDB 資料庫存取（如存取歷史紀錄），或將資料封裝並呼叫對應的 Python AI 模組處理，最終將運算結果統一格式後回傳至前端。

C. 資料庫架構 (Database Schema)：系統採用 MongoDB 進行資料持久化，以適應彈性的資料結構，核心設計三種 Models：

1. **Account (帳戶設定)：**儲存使用者基本資訊、關聯群組、擁有的任務 ID，以及透過表單蒐集而來的「個人時間偏好設定」。
2. **Group (群組紀錄)：**儲存該群組的基本資訊、成員名單，以及群組內的對話歷史紀錄，作為推論上下文之用。
3. **Task (任務內容)：**儲存任務的基本資訊，紀錄任務所有者、日期、標題、內容、耗時，以及「任務執行成效」（如是否完成、完成時的疲勞度、實際執行時間等數據），這些任務資訊將作為演算法的參考與統計資料。

D. AI 決策層 (AI Layer)：

1. **觸發管道：**針對「AI 助理」直接進行意圖分析；針對「群組任務」則設計了兩階段 AI 篩選機制，確保真正有排程意義的訊息才會進入核心演算法，大幅減少無效運算。

2. 分類器與處理模式 (Modes)：

Mode_1 – 具體排程：先將文字丟第一層時間提取的 LLM，裡面提供，明天/後天/下禮拜/下個月等等的中文模糊的時間範例，要求他完整提取出來，接者使用程式碼邏輯處理得出符合的區間，假設今天是 4/20 禮拜一，使用者希望下禮拜一下午五點要安排開會，則會提取出(2026-4-27, 2026-4-27)，並最後再使用一層 LLM 再提取日期、時間、內容。

Mode_2 - 模糊排程（資料充足）：先用使用模糊處理，提取是否有符合使用者填寫表單的任務的類別，像是工作、學習等等，接著使用「個人化演算法」，綜合過往習慣、RAG 向量檢索匹配來調整權重，算出較適合推薦的時段，最後使用與 Mode1 相同的 prompt 抽取日期、時間、內容、內容確保格式一致。

Mode_3 - 模糊排程（冷啟動）：系統轉換策略，利用 LLM 的零樣本推論能力，結合使用者的習慣表單進行排程建議，讓新使用者在資料不足的情況下也能享有個人化排程，接著將排程出的準確時間資料轉向 Mode 1，再次提取出日期、時間、內容，來確保格式一致。

Mode_4 – 非任務對話：系統判斷為無關排程的一般閒聊，不產出任何任務，請使用者重新輸入需求。

3. 排程演算法實作細節：

本系統的排程演算法採用「候選時段生成 + 多因子調分 + 記憶加權調整」的設計邏輯。首先，系統將一天切分為 48 個 30 分鐘的時間區塊，再依據任務需求時長，從使用者可用時段中產生所有合法的候選時間區塊。此設計可將原本模糊的時間安排問題轉換為可量化比較的區塊組合，作為後續評分的基礎。

a. 基礎分數計算機制：

每個候選時段都會先經過基礎分數計算，系統會根據使用者的歷史行為資料，建立兩類時間分布，其一為整體活動時間分布，反映使用者平常在哪些時段較常安排任務，其二為任務類型時間分布，反映特定類型任務（如讀書、作業、專題）過去較常出現的時間位置，演算法將兩者整合為基礎分數，其中任務類型分布給予較高權重，使系統不只考慮「常用時段」，也更重視「適合該任務類型的時段」。

b. 舒適度與疲勞成本調整：

在基礎分數之外，系統進一步加入使用者狀態與生理負擔的考量。若候選時段落在使用者表單設定的高專注區間（best focus periods），則提高其舒適度分數，反之，若時段過晚、時長過長，或與既有任務過於緊鄰，則增加疲勞懲罰。透過此機制，系統可避免只追求高頻歷史時段，卻忽略實際執行上的疲勞與壓力問題，使排程結果更貼近真實生活情境。

c. RAG 記憶加權機制：

為了提升排程的個人化程度，系統加入 RAG 記憶模組，從使用者過去相似情境的成功經驗中，檢索可用的調整資訊，其影響主要分為三層：第一，補充建議限制條件，例如偏好上午、避免熬夜、需要休息間隔等，第二，動態修正參數權重，例如提高舒適度權重、增加晚睡懲罰或調整疲勞倍率；第三，若過往成功紀錄集中於特定時間區段，則可直接對對應 block 進行加分。此機制使演算法不再僅依賴靜態規則，而能根據個人歷史經驗進行更精細的調整。

d. 排程模式與約束條件整合：

系統同時支援不同任務模式（normal、relaxed、push），以反映使用者在不同情境下的安排策略。例如 relaxed 模式會更重視舒適度並提高疲勞成本，push 模式則相對降低疲勞懲罰，使系統在趕工情境下能接受較高強度的安排，此外，像是 no late night、avoid long block、prefer morning 等限制條件，也會一併納入評分函數，進一步強化排程結果與使用者當下需求的一致性。

e. 候選時段選擇與衝突避免：

完成各候選時段評分後，系統會針對每個可排日期挑選前幾名高分候選時段，再綜合比較不同日期的結果。為避免任務過度集中於同一天，系統設計了日期使用懲罰，若某一日期已排入較多任務，則會降低其最終分數，另一方面，系統也會持續更新已使用的時間區塊，使後續任務不會與既有安排重疊，確保整體排程的可執行性與合理性。

f. **最終決策輸出：**

綜合上述因素後，系統會為每個候選時段計算最終分數，概念上可大致表示為：

$$\text{最終分數} = \text{基礎分數} + \text{舒適度加分} - \text{疲勞懲罰} + \text{RAG 調整} - \text{日期使用懲}$$

最終選出的結果，不是單純「有空的時間」，而是從使用者習慣、任務特性、狀態模式、記憶經驗與衝突限制等多面向綜合評估後，所產生的相對最佳排程建議。

六、結論

本專題成功實現了一套整合 LLM 與 RAG 的智慧排程代理系統且具備以下能力：

1. 群組聊天自動偵測使用者行程意圖，降低使用者手動輸入行程的負擔。
2. 個人化智慧排程，結合使用者歷史紀錄以及多項參數權重推薦貼合使用者習慣的時段推薦。
3. 持續學習機制，透過 RAG 的回饋使得系統可以隨者使用者完成更多任務而不斷累積知識，逐步提升排程品質。

七、參考文獻

<https://github.com/langchain-ai/rag-from-scratch>

<https://learning-axis.com/snippets/6888/>

<https://arxiv.org/abs/2509.25693>